Nadogradnja V03 na CQRS

Razvoj softvera I · Vježbe 4 · FIT UNMO 2026/27

Korak po korak, komponenta po komponenta

Svaka odgovornost iz akcije V03 dobija svoju komponentu u V04

V03 · akcija kontrolera

StudentiController(AppDbContext db) radi sve u jednoj metodi.

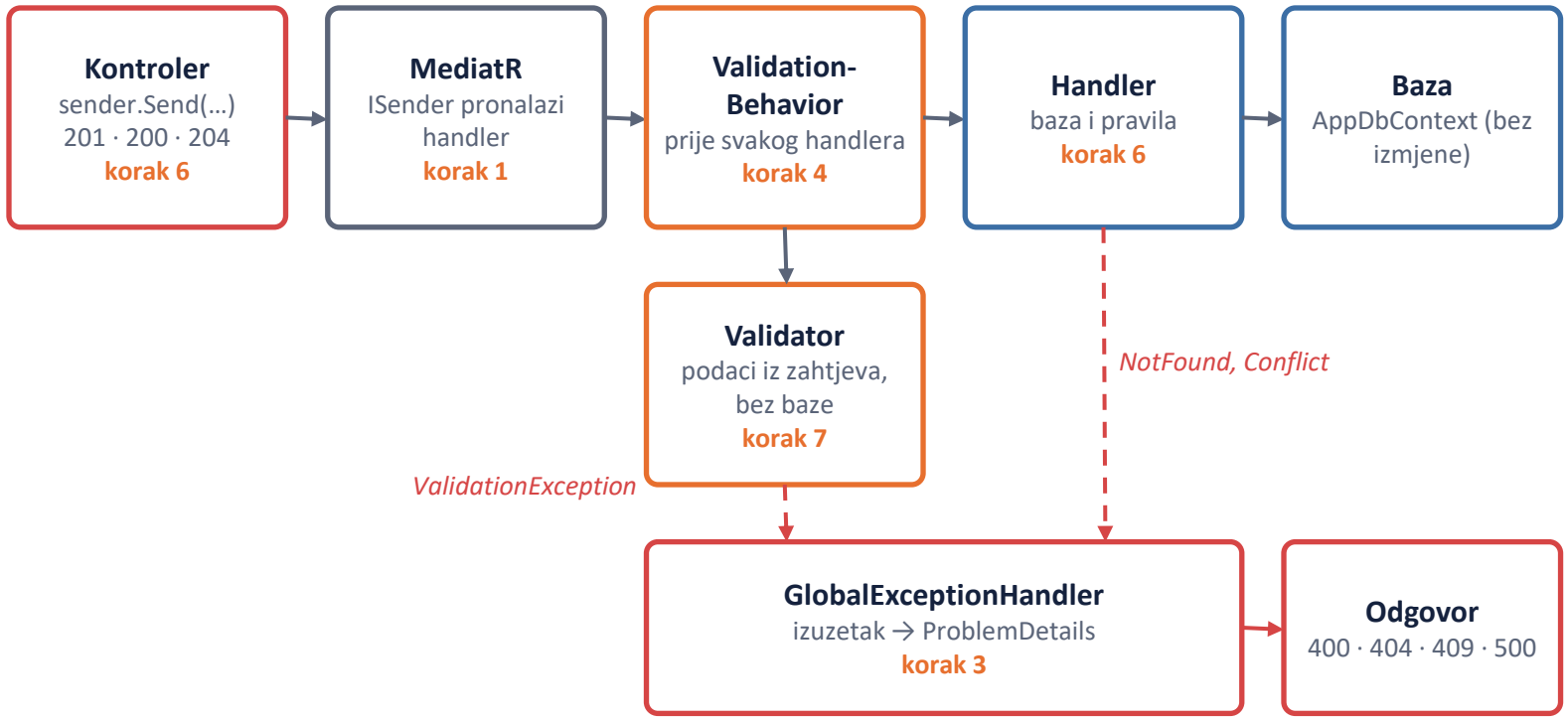
atributi na StudentDodajRequest
[Required], [RegularExpression] → 400

provjere nad bazom
broj indeksa, smjer, upisi → 400, 404, 409

rad s bazom
Add, SaveChangesAsync, Select

HTTP odgovor
NotFound(), Conflict(), CreatedAtAction

V04 · put zahtjeva kroz komponente



Kontroler

Prima zahtjev i šalje query ili command. Nema AppDbContext.

Validator

Provjerava samo podatke iz zahtjeva. Greška: 400.

Handler

Rad s bazom i pravila nad podacima: 404, 409.

Folderi prije i poslije: novi fajlovi su označeni brojem koraka

folderi V03

```
MojaApp.API/
├─ Controllers/
│   └─ StudentiController.cs    // baza, pravila, HTTP
├─ Data/AppDbContext.cs
├─ Dtos/
│   ├─ StudentDodajRequest.cs  // atributi [Required]
│   ├─ StudentUrediRequest.cs
│   ├─ StudentResponse.cs
│   └─ StudentDetaljiResponse.cs
├─ Migrations/
├─ Models/
└─ Program.cs
```

Šta se uklanja

- **StudentDodajRequest** i **StudentUrediRequest**
  Zamjenjuju ih command klase; atributi postaju pravila validatora (korak 7).
- AppDbContext iz kontrolera
  Kontroler dobija samo ISender (korak 8).

folderi V04

```
MojaApp.API/
├─ Common/
│   ├─ Behaviors/ValidationBehavior.cs      // 4
│   └─ Exceptions/                          // 2
│       ├─ NotFoundException.cs
│       └─ ConflictException.cs
├─ Controllers/StudentiController.cs      // 6, 7: samo ISender
├─ Data/AppDbContext.cs
├─ Dtos/
│   ├─ StudentResponse.cs                 // 5: Projekcija
│   └─ StudentDetaljiResponse.cs
├─ Middleware/GlobalExceptionHandler.cs   // 3
├─ Migrations/
├─ Models/
├─ Modules/Studenti/
│   ├─ Queries/Lista/   query, handler      // 6
│   ├─ Queries/PoId/    query, handler      // 6
│   ├─ Commands/Dodaj/  command, validator, handler   // 7
│   ├─ Commands/Uredi/  command, validator, handler   // 7
│   └─ Commands/Obrisi/ command, handler    // 7
├─ GlobalUsings.cs                        // 4
└─ Program.cs                             // 3, 4
```

Osam koraka: poslije svakog build i isti zahtjevi iz .http fajla

#	Korak	Fajlovi	Provjera poslije koraka
1	Paketi	MojaApp.API.csproj	build bez grešaka
2	Izuzeci	Common/Exceptions/NotFoundException.cs, ConflictException.cs	build bez grešaka
3	Greške → HTTP	Middleware/GlobalExceptionHandler.cs, Program.cs, appsettings.json	aplikacija se pokreće; statusi kao u V03
4	Validacija u pipelineu	Common/Behaviors/ValidationBehavior.cs, GlobalUsings.cs, Program.cs	aplikacija se pokreće; statusi kao u V03
5	Projekcija	Dtos/StudentResponse.cs	GET /Studenti: isti SQL kao u V03
6	Queries	Modules/Studenti/Queries/Lista i PoId, kontroler	GET /Studenti, /Studenti/1, /Studenti/999 (404)
7	Commands	Modules/Studenti/Commands/Dodaj, Uredi, Obrisi, kontroler	POST, PUT, DELETE: 201, 204, 400, 404, 409
8	Čišćenje	brisanje Dtos/StudentDodajRequest.cs i StudentUrediRequest.cs	kontroler i validatori bez AppDbContext

terminal nova grana, pa commit poslije svakog koraka

```
git switch -c v04-cqrs
git add .
git commit -m "V04 korak 1: paketi"
```

Radite na posebnoj grani

commit poslije svakog koraka

Ako nešto prestane raditi, uzrok je u jednom koraku: git diff pokazuje šta je izmijenjeno.

01

Infrastruktura: koraci 1–4

Paketi, izuzeci, globalni handler grešaka i validacija u MediatR pipelineu.

Postojeći kontroler u ovim koracima radi kao u V03.

Korak 1: tri NuGet paketa, iste verzije kao u starteru

cmd dotnet CLI, u folderu MojaApp.API (gdje je .csproj)

```
dotnet add package MediatR --version 12.5.0
dotnet add package FluentValidation --version 12.2.0
dotnet add package FluentValidation.DependencyInjectionExtensions --version 12.2.0
```

PMC ili Visual Studio: Tools → NuGet Package Manager → Package Manager Console, Default project: MojaApp.API

```
Install-Package MediatR -Version 12.5.0
Install-Package FluentValidation -Version 12.2.0
Install-Package FluentValidation.DependencyInjectionExtensions -Version 12.2.0
```

XML rezultat u MojaApp.API.csproj

```
<PackageReference Include="FluentValidation" Version="12.2.0" />
<PackageReference Include="FluentValidation.DependencyInjectionExtensions" Version="12.2.0" />
<PackageReference Include="MediatR" Version="12.5.0" />
```

MediatR 12.5.0

- ISender · IRequest

  Verzija 13 uvodi komercijalnu licencu; 12.5.0 je Apache 2.0.

FluentValidation 12.2.0

- AbstractValidator · RuleFor

  Pravila za ulazne podatke umjesto atributa.

…DependencyInjection-Extensions

- AddValidatorsFromAssembly

  Registruje sve validatore jednim redom koda (korak 4).

Provjera

build bez grešaka

Aplikacija radi kao u V03. Grafički: Manage NuGet Packages, ista imena i verzije.

Korak 2: izuzeci opisuju šta se desilo, a ne HTTP status

C# Common/Exceptions/NotFoundException.cs

```
namespace MojaApp.API.Common.Exceptions;

// Traženi zapis ne postoji. GlobalExceptionHandler ga pretvara u 404.
public class NotFoundException(string message) : Exception(message);
```

C# Common/Exceptions/ConflictException.cs

```
namespace MojaApp.API.Common.Exceptions;

// Zahtjev je ispravan, ali je u sukobu s postojećim podacima (duplikat, povezani zapisi). Postaje 409.
public class ConflictException(string message) : Exception(message);
```

folderi

```
MojaApp.API/
├─ Common/
│  └─ Exceptions/
│     ├─ NotFoundException.cs   // novo
│     └─ ConflictException.cs   // novo
├─ Controllers/
└─ …
```

V03, u kontroleru	V04, u handleru	status
`return NotFound();`	`throw new NotFoundException("…");`	404
`return Conflict("…");`	`throw new ConflictException("…");`	409

Primary constructor (C# 12): class NotFoundException(string message) : Exception(message) prosljeđuje poruku baznoj klasi Exception. Poruka se upisuje u polje detail odgovora.

Korak 3: GlobalExceptionHandler pretvara izuzetak u HTTP odgovor

C# Middleware/GlobalExceptionHandler.cs (1/2)

```
// Jedno mjesto koje izuzetak pretvara u HTTP odgovor u obliku ProblemDetails.
// Kontroleri i handleri zato nemaju try/catch ni return NotFound()/Conflict().
public sealed class GlobalExceptionHandler(ILogger<GlobalExceptionHandler> logger) : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext http, Exception ex, CancellationToken ct)
    {
        ProblemDetails problem = ex switch
        {
            // 400: greške po polju, isti oblik kao greške validacije iz V02/V03
            ValidationException v => new ValidationProblemDetails(
                v.Errors
                    .GroupBy(e => e.PropertyName)
                    .ToDictionary(g => g.Key, g => g.Select(e => e.ErrorMessage).ToArray()))
            {
                Status = StatusCodes.Status400BadRequest,
                Title = "Podaci nisu ispravni.",
            },
            NotFoundException => new ProblemDetails
            {
                Status = StatusCodes.Status404NotFound,
                Title = "Nije pronađeno.",
                Detail = ex.Message,
            },
```

ValidationException je iz FluentValidation (korak 4). GroupBy prema PropertyName daje „errors“: { "Ime": ["Ime je obavezno."] }.

Korak 3: GlobalExceptionHandler pretvara izuzetak u HTTP odgovor

C# Middleware/GlobalExceptionHandler.cs (2/2)

```
            …   // ConflictException → 409, isto kao NotFoundException
            // 500: detalji greške idu u log, ne klijentu
            _ => new ProblemDetails
            {
                Status = StatusCodes.Status500InternalServerError,
                Title = "Greška na serveru.",
            },
        };

        if (problem.Status == StatusCodes.Status500InternalServerError)
            logger.LogError(ex, "Neobrađena greška: {Metoda} {Putanja}", http.Request.Method, http.Request.Path);
        else
            logger.LogWarning("{Status} {Metoda} {Putanja}: {Poruka}",
                problem.Status, http.Request.Method, http.Request.Path, ex.Message);

        http.Response.StatusCode = problem.Status!.Value;
        await http.Response.WriteAsJsonAsync(problem, problem.GetType(), options: null,
            contentType: "application/problem+json", ct);
        return true; // izuzetak je obrađen
    }
}
```

return true znači da je izuzetak obrađen. Za 500 klijent dobija samo naslov; stack trace je u logu (LogError).

Korak 3: registracija u Program.cs i log u appsettings.json

C# Program.cs

```
// Izuzetak iz handlera postaje HTTP odgovor (400, 404, 409, 500) na jednom mjestu.
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();
builder.Services.AddProblemDetails();

var app = builder.Build();

// Configure the HTTP request pipeline.
app.UseExceptionHandler();
```

JSON appsettings.json

```
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      // Greške loguje GlobalExceptionHandler (404/409 kao upozorenje, 500 sa stack traceom),
      // pa ugrađeni middleware ne treba za svaki 404 ispisivati cijeli stack trace.
      "Microsoft.AspNetCore.Diagnostics.ExceptionHandlerMiddleware": "None"
```

Bez AddProblemDetails

- aplikacija se **ne pokreće**
  InvalidOperationException (izmjereno, .NET 8).

Log

- u log piše GlobalExceptionHandler
  Inače ugrađeni middleware za svaki izuzetak ispisuje i stack trace.

Provjera

aplikacija se pokreće

Statusi kao u V03: kontroler još vraća NotFound().

Korak 4: ValidationBehavior pokreće validatore prije svakog handlera

C# Common/Behaviors/ValidationBehavior.cs (isti kod je u starteru)

```
// Korak u MediatR pipelineu: prije handlera pokreće sve validatore za taj zahtjev.
// Ako ijedno pravilo ne prođe, handler se ne poziva, nego se baca ValidationException (postaje 400).
public sealed class ValidationBehavior<TRequest, TResponse>(IEnumerable<IValidator<TRequest>> validators)
    : IPipelineBehavior<TRequest, TResponse>
    where TRequest : notnull
{
    public async Task<TResponse> Handle(TRequest request, RequestHandlerDelegate<TResponse> next, CancellationToken ct)
    {
        List<ValidationFailure> greske = [];

        var context = new ValidationContext<TRequest>(request);
        foreach (IValidator<TRequest> validator in validators)
        {
            ValidationResult rezultat = await validator.ValidateAsync(context, ct);
            greske.AddRange(rezultat.Errors);
        }

        if (greske.Count != 0)
            throw new ValidationException(greske);

        return await next();
    }
}
```

validators: svi validatori za TRequest (DI ih prosljeđuje konstruktoru). Ako za zahtjev nema validatora, lista je prazna i odmah se poziva next().

Korak 4: MediatR, validatori i ValidationBehavior u Program.cs

C# Program.cs · ispod: izmjereno na gotovom rješenju V04 bez jedne od ovih linija

```
builder.Services.AddControllers(options =>
{
    // Pravila za ulazne podatke su u validatorima (FluentValidation), ne u atributima.
    // Bez ovoga bi ASP.NET svaki string koji nije nullable tretirao kao [Required] i vratio 400 prije validatora.
    options.SuppressImplicitRequiredAttributeForNonNullableReferenceTypes = true;
});

// MediatR: pronalazi sve handlere (IRequestHandler) u ovom projektu.
builder.Services.AddMediatR(cfg => cfg.RegisterServicesFromAssembly(typeof(Program).Assembly));

// FluentValidation: registruje sve validatore (AbstractValidator) iz ovog projekta.
builder.Services.AddValidatorsFromAssembly(typeof(Program).Assembly);

// Prije svakog handlera MediatR pokreće validatore za taj zahtjev.
builder.Services.AddTransient(typeof(IPipelineBehavior<,>), typeof(ValidationBehavior<,>));
```

Bez AddMediatR

- svaki zahtjev → **500**

  Log: Unable to resolve service for type 'MediatR.ISender'.

Bez AddTransient(…Behavior)

- prazno ime · indeks „123“ → **201**

  Validatori se ne pokreću; u bazu se upisuju neispravni podaci o studentu.

Bez SuppressImplicitRequired…

- "ime": null → **400** iz ASP.NET Core

  Prije validatora vraća se „The Ime field is required.“ u drugom obliku odgovora.

Korak 4: GlobalUsings i redoslijed poziva kroz MediatR

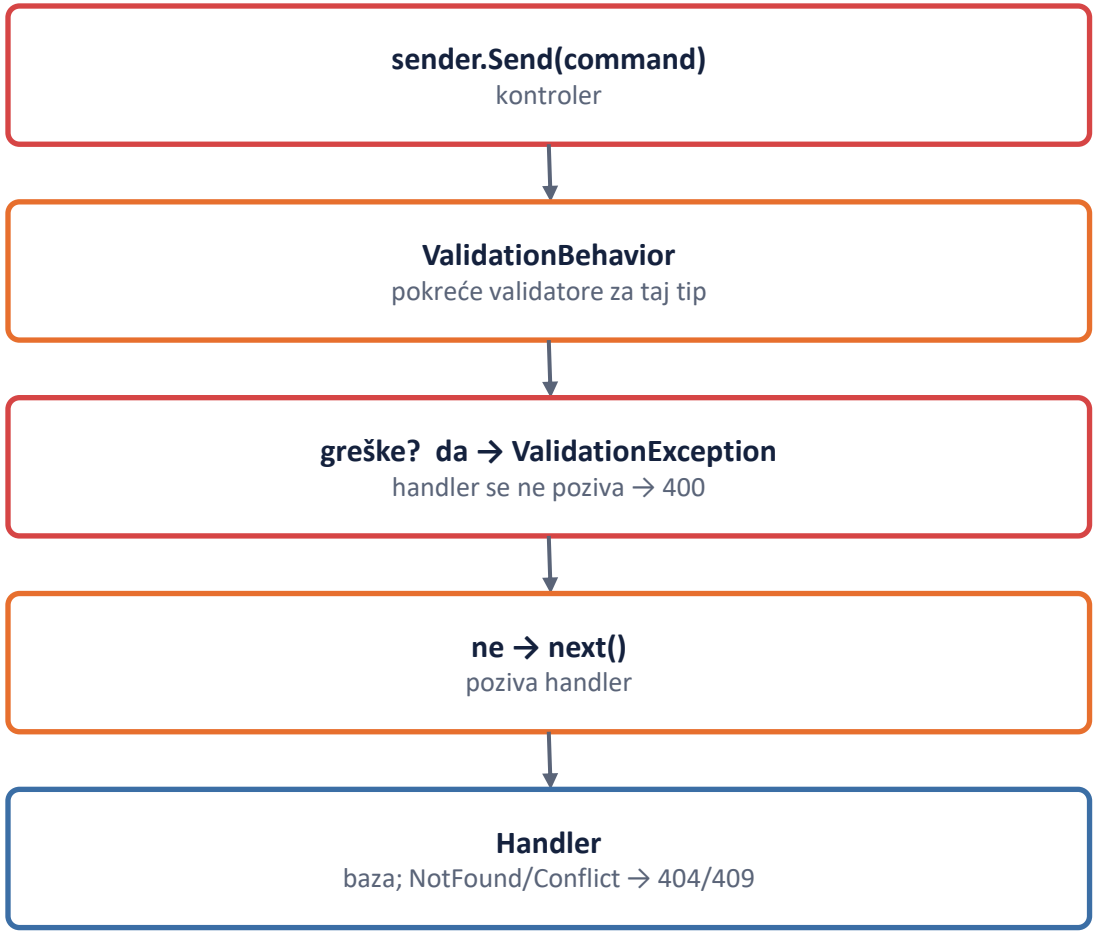
C# GlobalUsings.cs

```
global using FluentValidation;
global using MediatR;
global using Microsoft.EntityFrameworkCore;
global using MojaApp.API.Common.Exceptions;
global using MojaApp.API.Data;
global using MojaApp.API.Dtos;
global using MojaApp.API.Models;
```

Isti princip kao GlobalUsings u starteru. Namespace MojaApp.API.Common.Exceptions postoji od koraka 2.

Provjera

- build i pokretanje
  Statusi kao u V03. Nijedan zahtjev još ne ide kroz MediatR.



02

Studenti: koraci 5–8

Projekcija, queries, commands i čišćenje.

Kontroler se prebacuje akciju po akciju; poslije svake akcije isti zahtjevi iz .http fajla.

Korak 5: projekcija na jednom mjestu, kao Expression

C# V03 · StudentiController.cs, GetAll i Dodaj

```
.Select(x => new StudentResponse
{
    Id = x.Id,
    ImePrezime = x.Ime + " " + x.Prezime,
    BrojIndeksa = x.BrojIndeksa,
    DatumRodjenja = x.DatumRodjenja,
    Smjer = x.Smjer != null ? x.Smjer.Naziv : null, // LEFT JOIN Smjerovi
})
```

Expression, ne Func

- EF Core čita stablo izraza i pravi SQL

  Izmjereno: s Func EF Core čita sve kolone bez JOIN-a, a Smjer je null.

- u GetAll i Dodaj: **.Select(StudentResponse.Projekcija)**

C# V04 · Dtos/StudentResponse.cs (+ using System.Linq.Expressions;)

```
// Projekcija na jednom mjestu. Expression (a ne obična metoda) da EF Core može od nje napraviti SQL:
// SELECT samo potrebnih kolona i LEFT JOIN na Smjerovi, kao kad je Select napisan direktno u upitu.
public static readonly Expression<Func<Student, StudentResponse>> Projekcija = x => new StudentResponse
{
    Id = x.Id,
    ImePrezime = x.Ime + " " + x.Prezime,
    BrojIndeksa = x.BrojIndeksa,
    DatumRodjenja = x.DatumRodjenja,
    Smjer = x.Smjer != null ? x.Smjer.Naziv : null,
};
```

Provjera: GET /Studenti. U konzoli je isti SQL kao u V03 (LEFT JOIN s tabelom Smjerovi). Ovaj korak se može uraditi u kontroleru V03, prije uvođenja handlera.

Korak 6: prvi query vraća listu, a tijelo GetAll prelazi u handler

C# Modules/Studenti/Queries/Lista/ListaStudenataQuery.cs

```
public class ListaStudenataQuery : IRequest<List<StudentResponse>>
{
    public string? Pretraga { get; set; } // ime, prezime ili broj indeksa
}
```

C# Modules/Studenti/Queries/Lista/ListaStudenataQueryHandler.cs (bez praznih redova)

```
public class ListaStudenataQueryHandler(AppDbContext db)
    : IRequestHandler<ListaStudenataQuery, List<StudentResponse>>
{
    public async Task<List<StudentResponse>> Handle(ListaStudenataQuery request, CancellationToken ct)
    {
        IQueryable<Student> upit = db.Studenti;
        if (!string.IsNullOrWhiteSpace(request.Pretraga))
            upit = upit.Where(x => x.Ime.Contains(request.Pretraga)
                                || x.Prezime.Contains(request.Pretraga)
                                || x.BrojIndeksa.Contains(request.Pretraga));
        return await upit
            .OrderBy(x => x.Prezime).ThenBy(x => x.Ime)
            .Select(StudentResponse.Projekcija) // projekcija s jednog mjesta, SQL ostaje isti kao u V03
            .ToListAsync(ct);
    }
}
```

Korak 6: kontroler šalje query, a handler baca NotFoundException

C# Controllers/StudentiController.cs · V04

```
// GET /Studenti?pretraga=ana
[HttpGet]
public Task<List<StudentResponse>> GetAll([FromQuery] ListaStudenataQuery query, CancellationToken ct)
    => sender.Send(query, ct);

// GET /Studenti/5: student s upisanim predmetima i ocjenama (404 ako ne postoji)
[HttpGet("{id:int}")]
public Task<StudentDetaljiResponse> GetById(int id, CancellationToken ct)
    => sender.Send(new StudentPoIdQuery { Id = id }, ct);
```

C# V03

```
if (student == null)
    return NotFound();
```

C# V04 · Queries/PoId/StudentPoIdQueryHandler.cs

```
// Umjesto return NotFound(): izuzetak, a GlobalExceptionHandler od njega pravi 404.
if (student == null)
    throw new NotFoundException($"Student (Id = {request.Id}) ne postoji.");

return student;
```

Zahtjev	Status	Odgovor (izmjereno)
GET /Studenti, GET /Studenti/1	200	isto kao u V03
GET /Studenti/999	404	{ "title": "Nije pronađeno.", "status": 404, "detail": "Student (Id = 999) ne postoji." }

Korak 7: command zamjenjuje request, atributi postaju pravila validatora

C# V03 · Dtos/StudentDodajRequest.cs (bez praznih redova)

```
public class StudentDodajRequest
{
    [Required, StringLength(50)]
    public string Ime { get; set; } = "";
    [Required, StringLength(50)]
    public string Prezime { get; set; } = "";
    [Required, RegularExpression(@"^IB\d{6}$", ErrorMessage = "Broj indeksa mora biti u formatu IB123456.")]
    public string BrojIndeksa { get; set; } = "";
    public DateOnly? DatumRodjenja { get; set; }
    public int? SmjerId { get; set; } // smjer nije obavezan
}
```

C# V04 · Modules/Studenti/Commands/Dodaj/DodajStudentaCommand.cs

```
public class DodajStudentaCommand : IRequest<StudentResponse>
{
    public string Ime { get; set; } = "";
    public string Prezime { get; set; } = "";
    public string BrojIndeksa { get; set; } = "";
    public DateOnly? DatumRodjenja { get; set; }
    public int? SmjerId { get; set; } // smjer nije obavezan
}
```

Id i DatumKreiranja postavlja server, pa ih nema ni u commandu. Naziv klase slijedi obrazac iz startera: glagol + entitet + Command (CreateProductCategoryCommand); folder Commands/Dodaj sadrži command, validator i handler.

Korak 7: validator provjerava samo podatke iz zahtjeva, bez baze

C# Commands/Dodaj/DodajStudentaCommandValidator.cs

```
public class DodajStudentaCommandValidator : AbstractValidator<DodajStudentaCommand>
{
    public DodajStudentaCommandValidator()
    {
        RuleFor(x => x.Ime)
            .NotEmpty().WithMessage("Ime je obavezno.")
            .MaximumLength(50).WithMessage("Ime može imati najviše 50 znakova.");

        RuleFor(x => x.Prezime)
            .NotEmpty().WithMessage("Prezime je obavezno.")
            .MaximumLength(50).WithMessage("Prezime može imati najviše 50 znakova.");

        RuleFor(x => x.BrojIndeksa)
            .Matches(@"^IB\d{6}$").WithMessage("Broj indeksa mora biti u formatu IB123456.");
    }
}
```

[Required] → NotEmpty() · [StringLength(50)] → MaximumLength(50)

[RegularExpression] → Matches(…), ErrorMessage → WithMessage(…).

Validator: oblik podataka → 400

Handler: stanje baze → 404 (smjer ne postoji), 409 (broj indeksa postoji).

Korak 7: handler radi s bazom i baca izuzetke umjesto HTTP odgovora

C# Commands/Dodaj/DodajStudentaCommandHandler.cs · Handle; klasa: DodajStudentaCommandHandler(AppDbContext db)

```
// Format podataka je već provjerio validator. Ovdje su pravila koja zavise od stanja baze.
if (request.SmjerId != null && !await db.Smjerovi.AnyAsync(x => x.Id == request.SmjerId, ct))
    throw new NotFoundException($"Smjer (Id = {request.SmjerId}) ne postoji."); // 404

if (await db.Studenti.AnyAsync(x => x.BrojIndeksa == request.BrojIndeksa, ct))
    throw new ConflictException($"Student s brojem indeksa {request.BrojIndeksa} već postoji."); // 409

Student student = new()
{
    Ime = request.Ime,
    …
    SmjerId = request.SmjerId,
};

db.Studenti.Add(student);
await db.SaveChangesAsync(ct);

// Ista projekcija kao u listi: nema ponavljanja koda iz V03.
StudentResponse response = await db.Studenti
    .Where(x => x.Id == student.Id)
    .Select(StudentResponse.Projekcija)
    .FirstAsync(ct);

return response;
```

Korak 7: akcija Dodaj u kontroleru i provjera svih ishoda

C# Controllers/StudentiController.cs · V04

```
[HttpPost]
public async Task<ActionResult<StudentResponse>> Dodaj(
    DodajStudentaCommand command, CancellationToken ct)
{
    StudentResponse student = await sender.Send(command, ct);
    return CreatedAtAction(nameof(GetById), new { id = student.Id }, student); // 201
}
```

POST /Studenti	Status	Odakle	Odgovor (izmjereno)
ispravni podaci, smjerId: 1	201	kontroler	{ "id": 3, "imePrezime": "Lejla Lejlić", …, "smjer": "Softverski inženjering" }
ime "", brojIndeksa "123"	400	validator	"errors": { "Ime": ["Ime je obavezno."], "BrojIndeksa": ["Broj indeksa mora biti u formatu IB123456."] }
smjerId: 77	404	handler	"detail": "Smjer (Id = 77) ne postoji."
brojIndeksa "IB260001" (postoji)	409	handler	"detail": "Student s brojem indeksa IB260001 već postoji."

Nepostojeći smjer: V03 je vraćao **400**, V04 vraća **404**

Provjera je u handleru, kao u starteru: CreateProductCommandHandler za nepostojeću kategoriju baca MarketNotFoundException.

Korak 7: Uredi po istom obrascu, a Id dolazi iz rute

C# Modules/Studenti/Commands/Uredi/UrediStudentaCommand.cs (+ using System.Text.Json.Serialization;)

```
public class UrediStudentaCommand : IRequest
{
    [JsonIgnore] // Id dolazi iz rute (PUT /Studenti/5), ne iz tijela zahtjeva
    public int Id { get; set; }
    public string Ime { get; set; } = "";
    public string Prezime { get; set; } = "";
    public DateOnly? DatumRodjenja { get; set; }
    public int? SmjerId { get; set; }
}
```

C# Controllers/StudentiController.cs

```
[HttpPut("{id:int}")]
public async Task<IActionResult> Uredi(int id, UrediStudentaCommand command, CancellationToken ct)
{
    command.Id = id; // Id iz rute ima prednost
    await sender.Send(command, ct);
    return NoContent(); // 204
}
```

Validator za Uredi ima ista pravila kao za Dodaj, bez broja indeksa. Handler: student ne postoji → 404, smjer ne postoji → 404. Provjera: PUT /Studenti/2 → 204, /Studenti/99 → 404, prazno ime → 400.

Korak 7: Obriši nema validatora, a oba pravila su u handleru

C# Commands/Obrisi/ObrisiStudentaCommand.cs

```
public class ObrisiStudentaCommand : IRequest
{
    public int Id { get; set; }
}
```

C# Controllers/StudentiController.cs

```
[HttpDelete("{id:int}")]
public async Task<IActionResult> Obrisi(int id, CancellationToken ct)
{
    await sender.Send(new ObrisiStudentaCommand { Id = id }, ct);
    return NoContent(); // 204
}
```

C# Commands/Obrisi/ObrisiStudentaCommandHandler.cs · metoda Handle

```
Student student = await db.Studenti.FirstOrDefaultAsync(x => x.Id == request.Id, ct)
    ?? throw new NotFoundException($"Student (Id = {request.Id}) ne postoji."); // 404

// Upis ima OnDelete(Restrict). Provjera ovdje daje jasnu poruku (409) umjesto greške iz baze (500).
if (await db.Upisi.AnyAsync(x => x.StudentId == request.Id, ct))
    throw new ConflictException("Student ima upisane predmete i ne može se obrisati."); // 409

db.Studenti.Remove(student);
await db.SaveChangesAsync(ct);
```

Provjera: DELETE /Studenti/1 → 409 (ima upise), DELETE /Studenti/999 → 404.

Korak 8: stari DTO-i se brišu, a kontroler ima samo ISender

C# Controllers/StudentiController.cs

```
// Tanak kontroler: prima HTTP zahtjev, šalje query/command kroz MediatR i vraća rezultat.
// Nema AppDbContext, nema provjera, nema NotFound()/Conflict(): to je u handlerima i GlobalExceptionHandler-u.
[ApiController]
[Route("[controller]")]
public class StudentiController(ISender sender) : ControllerBase
```

folderi

```
Dtos/
├─ StudentDodajRequest.cs    // obrisati
├─ StudentUrediRequest.cs    // obrisati
├─ StudentResponse.cs        // korak 5
└─ StudentDetaljiResponse.cs
```

- Predmeti i Upisi: zadaci
  Isti postupak (slajd „Zadaci“).

terminal PowerShell, u folderu MojaApp.API: prazan izlaz

```
Select-String -Path Controllers\StudentiController.cs,
  Modules\Studenti\Commands\*\*Validator.cs -Pattern "AppDbContext db"
```

Završna provjera

- build bez upozorenja · svi zahtjevi iz .http
  Statusi kao u V03, osim nepostojećeg smjera (404). Commit i pull request.

03

Postupak, česte greške i zadaci

Isti postupak za svaku akciju, izmjerene greške pri registraciji i preslikavanje na starter.

Postupak za jednu akciju: pet koraka, uvijek istim redom

1 Klasa
Query (čita) ili Command (mijenja)
IRequest<T> ili IRequest; folder Modules/<Entitet>/…

2 Validator
ako zahtjev ima tijelo (JSON)
AbstractValidator<T>, bez AppDbContext; Obriši, Lista i PoId ga nemaju

3 Handler
tijelo stare akcije
return NotFound/Conflict → throw

4 Kontroler
sender.Send(…)
201/204 i dalje vraća kontroler

5 Provjera
isti zahtjevi iz .http
build, statusi, commit

U V03 (kontroler)	U V04	Gdje
`[Required], [StringLength], [RegularExpression]`	`NotEmpty(), MaximumLength(), Matches()`	validator
`return NotFound();`	`throw new NotFoundException("…");`	handler
`return Conflict("…");`	`throw new ConflictException("…");`	handler
`ModelState.AddModelError(…); return ValidationProblem(…);`	`ValidationException iz ValidationBehavior`	pipeline
`db.… u akciji kontrolera`	`db.… u handleru`	handler
`.Select(x => new StudentResponse { … }) dvaput`	`.Select(StudentResponse.Projekcija)`	Dtos

Česte greške: simptom, uzrok i korak u kojem se ispravljaju

Simptom (izmjereno)	Uzrok	Korak
Svaki zahtjev na /Studenti vraća 500. U logu: Unable to resolve service for type 'MediatR.ISender' while attempting to activate 'MojaApp.API.Controllers.StudentiController'.	nema AddMediatR(…)	4
Aplikacija se ne pokreće. InvalidOperationException: An error occurred when configuring the exception handler middleware.	UseExceptionHandler() bez AddProblemDetails()	3
POST s praznim imenom i indeksom „123“ vraća 201 i upisuje neispravne podatke o studentu.	ValidationBehavior nije registrovan	4
"ime": null vraća 400 „The Ime field is required.“ s naslovom „One or more validation errors occurred.“	nema SuppressImplicitRequired…	4
U listi studenata Smjer je null; u SQL-u nema JOIN-a i čitaju se sve kolone.	Func<Student, StudentResponse> umjesto Expression<…>	5

Mjerenje 28.10.2026, LocalDB. Redovi 1–4: kopija rješenja bez jedne linije u Program.cs, isti zahtjev kao u .http fajlu. Red 5: ista lista s Func umjesto Expression.

MojaApp i starter: isti obrazac, drugi raspored foldera

MojaApp (V04)	Starter za seminarski rad	Uloga
`Controllers/StudentiController.cs`	`Market.API/Controllers/ProductCategoriesController.cs`	tanak kontroler koji koristi ISender
`Modules/Studenti/Queries/PoId/`	`Market.Application/Modules/Catalog/ProductCategories/Queries/GetById/`	query i handler
`Modules/Studenti/Commands/Dodaj/`	`Market.Application/Modules/Catalog/ProductCategories/Commands/Create/`	command, validator i handler
`Common/Behaviors/ValidationBehavior.cs`	`Market.Application/Common/Behaviors/ValidationBehavior.cs`	validacija prije handlera
`Common/Exceptions/`	`Market.Application/Common/Exceptions/`	NotFound, Conflict (starter ima i 401, 403)
`Middleware/GlobalExceptionHandler.cs`	`Market.Infrastructure/Common/MarketExceptionHandler.cs`	izuzetak → HTTP odgovor
`Data/AppDbContext.cs`	`Market.Infrastructure/Database/DatabaseContext.cs` (IAppDbContext)	pristup bazi
`Models/`	`Market.Domain/Entities/`	entiteti

Razlika: handleri u starteru dobijaju IAppDbContext (interfejs), pa Application ne zavisi od Infrastructure. Starter vraća vlastiti ErrorDto umjesto ProblemDetails.

Zadaci: Predmeti i Upisi po istom postupku

1 · GET /Predmeti i /Predmeti/{id}

dva queryja

PredmetResponse.Projekcija kao Expression, s brojem upisanih studenata. Statusi: 200 · 404.

2 · POST /Predmeti

command i validator

Naziv, šifra, ECTS 1–30. Ista šifra: ConflictException. Statusi: 201 · 400 · 409.

3 · PUT /Upisi/{id}/ocjena

command i validator

Ocjena 6–10 u validatoru; UpisId iz rute ([JsonIgnore]). Statusi: 204 · 400 · 404.

4 · DELETE /Predmeti/{id}

command i handler

Predmet s upisima se ne briše. Bez try/catch i bez return Conflict() u kontroleru. Statusi: 204 · 404 · 409.

5 · Dodatno

LoggingBehavior

Drugi IPipelineBehavior: zapisuje naziv zahtjeva i trajanje u log.

Za svaki zadatak

issue · grana · pull request

Kao u V02 i V03: commit po koraku postupka.

Isti endpointi, svaka odgovornost na svom mjestu

Kontroler

sender.Send(…)

Prima HTTP zahtjev i vraća 200, 201 ili 204.

Validator

podaci iz zahtjeva

Bez baze. Greška: 400 prije handlera.

Handler

baza i pravila

NotFoundException → 404, ConflictException → 409.

Materijali: V04 - MojaApp rješenje.zip · V04-put-zahtjeva-kroz-slojeve.html (živi primjer: put zahtjeva korak po korak)